Ansible desde Cero: Automatiza Linux y Servidores

Cursos PIT - 10ma Edicion



Instructor: Cristian Adrian Diaz Garcia

Fecha: 20 de agosto de 2026

UNIVERSIDAD NACIONAL DE INGENIERÍA

CURSOS GRATUITOS

OTI UNI

TRANSFORMACIÓN digital

SESIÓN 4

VARIABLES, FACTS, CONDICIONALES Y TEMPLATES

SESIÓN 4

- Variables en Ansible: tipos, ubicación y uso dentro del proyecto.
- Facts: información automática del sistema para adaptar configuraciones.
- Condicionales con when y repetición de tareas con loop.
- Templates Jinja2 para generar configuraciones dinámicas según el entorno.

Facts

¿Qué son los Facts?

Definición
*Los **Facts** son variables que Ansible **descubre automáticamente** al conectarse a un host, antes de ejecutar cualquier tarea.*

Información que recoge Ansible:

- Sistema operativo y versión.
- Dirección IP de cada interfaz de red.
- Cantidad de CPUs y RAM disponible.
- Hostname, arquitectura, tiempo
- activo.

```
[lisa@drdevl ~]$ ansible hqdevl.tekneed.com -m setup
hqdevl.tekneed.com | SUCCESS => {
    "ansible_facts": {
        "ansible_all_ipv4_addresses": [
            "192.168.170.141",
            "192.168.122.1"
        ],
        "ansible_all_ipv6_addresses": [
            "fe80::ad03:2f76:22a3:99dc"
        ],
        "ansible_apparmor": {
            "status": "disabled"
        },
        "ansible_architecture": "x86_64",
        "ansible_bios_date": "05/19/2017",
```

Explorando los Facts del Sistema

Ver todos los facts de un host:

```
ansible ubuntu-node -m setup
```

Filtrar por patrón:

```
ansible ubuntu-node -m setup -a "filter=ansible_memory*"
ansible ubuntu-node -m setup -a "filter=ansible_*ipv4*"
```

Deshabilitar la recopilación de facts

En playbooks que no necesitan facts (o cuando son lentos en hosts remotos), puedes desactivarlos:

```
- hosts: all
  gather_facts: false # Ahorra 1-2 segundos por host en playbooks grandes
```

Variables más usadas

Variable	Ejemplo de valor
`ansible_facts['os_family']`	Debian, RedHat
`ansible_facts['distribution']`	Ubuntu, Rocky
`ansible_facts['distribution_version']`	22.04, 8.7
`ansible_facts['hostname']`	ubuntu-node
`ansible_facts['processor_vcpus']`	4
`ansible_facts['memtotal_mb']`	8192

Usando Facts en Playbooks

Mostrar IP y SOde cada servidor

```
- name: Mostrar información del sistema
  hosts: all
  tasks:
    - name: Info del host
      debug:
        msg: >
          Host: {{ ansible_hostname }},
          IP: {{ ansible_default_ipv4.address }},
          SO: {{ ansible_distribution }} {{ ansible_distribution_version }}
```

Los facts se usan igual que cualquier variable: con dobles llaves {{ }}.

Crear facts personalizados (set_fact)

set_fact *es un módulo que te permite crear o modificar variables dinámicamente en tiempo de ejecución (mientras tu playbook se está ejecutando).*

- Alcance (Scope): Las variables están vinculadas al host específico en el que se ejecutan.
- Persistencia: Esta estará disponible para todas las tareas y plays la ejecucion del playbook.
- Sobrescritura: Puedes usarlo para sobrescribir variables que hayan sido definidas previamente.

```
- name: Calcular memoria disponible para la JVM
  set_fact:
    jvm_heap_mb: "{{ (ansible_facts['memtotal_mb'] * 0.7) | int }}"

- name: Lanzar aplicación Java con heap calculado
  command: "java -Xmx{{ jvm_heap_mb }}m -jar app.jar"
```

When

Condicionales con *when*

¿Para qué sirven?
Ejecutar una tarea solo si se cumple unacondición. Combinados con Facts, permiten que el mismo playbook funcione en Ubuntu y CentOS.

```yaml
- name: Instalar en Debian/Ubuntu
  apt:
    name: nginx
    state: present
  when: ansible_os_family == "Debian"
```

```yaml
---
- name: install Apache Web-Server
  hosts: all
  tasks:
  - name: Install Apache on Rocky Linux Server
    dnf:
      name: httpd
      state: present
    become: yes
    when: ansible_os_family == "RedHat"
  - name: Install Apache2 on Ubuntu Server
    apt:
      name: apache2
      state: present
    become: yes
    when: ansible_os_family == "Debian"
```

Ejemplos de condiciones comunes

```
# Verificar versión del SO
- name: Tarea solo para Ubuntu 20.04
  debug:
    msg: "Este es Ubuntu 20.04"
  when: ansible_facts['distribution_version'] == "20.04"

# Verificar si existe un archivo
- name: Configurar backup solo si existe el directorio
  command: /usr/local/bin/backup.sh
  when: backup_dir.stat.exists
```

Ejemplos de condiciones comunes

```
# Múltiples condiciones (AND)
- name: Instalar solo en servidores de producción con Ubuntu
  apt:
    name: monitoring-agent
  when:
    - ansible_facts['distribution'] == "Ubuntu"
    - ansible_facts['environment'] == "production"

# Condición OR
- name: Instalar en Debian o Ubuntu
  apt:
    name: nginx
  when: ansible_facts['distribution'] == "Debian" or ansible_facts['distribution'] ==
"Ubuntu"
```

Loop

Repetición de Tareas con loop

¿Cuándo usar loop?
Ejecutar la misma acción sobre una lista de elementos (paquetes, usuarios, directorios) sin repetir código.

Sin loop (repetitivo)

```
- apt:
    name: curl
    state: present
- apt:
    name: git
    state: present
- apt:
    name: vim
    state: present
```

Con loop (simplificado)

```
- name: Instalar paquetes
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - curl
    - git
    - vim
```

Loops con diccionarios y condición

Loops con diccionarios

```
- name: Crear múltiples usuarios
  user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
    shell: /bin/bash
  loop:
    - name: 'alice'
      groups: 'sudo,developers'
    - name: 'bob'
      groups: 'developers'
    - name: 'charlie'
      groups: 'sudo'
```

Loop con condición

```
- name: Instalar paquetes solo en
servidores web
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - php-fpm
    - mysql-client
  when: "'servers' in group_names"
```

loop_control

Definición
es una directiva que se utiliza junto con loop para modificar y personalizar el comportamiento de un bucle.

Los 4 poderes:

- **loop_var:** Cambia el nombre de la variable (evita usar siempre item). Ideal para bucles anidados.
- **label:** Limpia la consola ocultando datos masivos o sensibles.
- **index_var:** Crea un contador (0, 1, 2...) para saber en qué iteración estás.
- **pause:** Agrega segundos de espera entre iteraciones (útil para APIs o reinicios).

```
- name: Instalar paquetes
  apt:
    name: "{{ pkg.name }}"
    state: "{{ pkg.state }}"
  loop:
    - name: nginx
      state: present
    - name: apache2
      state: absent
  loop_control:
    loop_var: pkg
    label: "{{ pkg.name }}"
    pause: 1
    index_var: idx
```

Templates Jinja2

Tags

Los tags son etiquitas que te permiten ejecutar solo ciertas tasks del playbook sin ejecutarlo completo.

```yaml
tasks:
 - name: Instalar paquetes base
 apt:
 name: "{{ item }}"
 state: present
 loop:
 - git
 - curl
 tags:
 - install
 - base
```

Ejecutar solo las tasks con tag "install"

```
ansible-playbook site.yml --tags install
```

Ejecutar solo "security" y "firewall"

```
ansible-playbook site.yml --tags "install,base"
```

Ejecutar TODO excepto las tasks con tag "config"

```
ansible-playbook site.yml --skip-tags base
```

Listar todos los tags disponibles

```
ansible-playbook site.yml --list-tags
```

always *siempre se ejecuta*
never *nunca se ejecuta a menos que se especifique*

¿Qué son los Templates Jinja2?

Definición
Los Templates son archivos de texto con marcadores que Ansible reemplaza con variables
reales antes de enviarlos al servidor.

Ventajas:

- Un solo archivo para múltiples entornos.
- Variables, condicionales y loops dentro del archivo.
- Motor Jinja2 (mismo que Flask, Django).



Extensión .j2 — se guardan en templates/

Sintaxis Jinja2 Esencial

Variables	Condicionales	Bucles	Comentarios
`{{ nombre_var }}`	`{ % if condicion %}`	`{ % for x in lista %}`	`{# comentario #}`
Inserta el valor de la variable	*Bloque condicional*	*Itera sobre una lista*	*No se renderiza*

Ejemplo: Template de Nginx

Archivo: templates/nginx.conf.j2

```
server {
    listen {{ puerto_http }};
    server_name {{ ansible_hostname }};

    root {{ directorio_web }};
    index index.html;

    {% if entorno == 'produccion' %}
    access_log /var/log/nginx/access_prod.log;
    error_log /var/log/nginx/error_prod.log;
    {% else %}
    access_log /var/log/nginx/access_dev.log;
    {% endif %}
}
```

Aplicando el Template con la Tarea

Tarea en el playbook

```
- name: Generar configuración de Nginx
  template:
    src: templates/nginx.conf.j2
    dest: /etc/nginx/sites-available/default
    owner: root
    group: root
    mode: '0644'
  notify: Reiniciar Nginx
```

Flujo Completo: Variables + Facts + Templates

Variables	Facts	when / loop	Template .j2
Definidas en vars:, group_vars/ o host_vars/	*Descubiertos automáticamente: OS, IP, RAM...*	*Lógica condicional y repetición de tareas*	*Configuraciones dinámicas por host y entorno*

Un playbook inteligente que adapta su comportamiento **automáticamente** según el servidor, el entorno y los datos del sistema.

Gracias!!!

